SQL:
A COMMERCIAL DATABASE LANGUAGE

Data Change Statements,
Views,
Complex Constraints

Outline

1. Introduction
2. Data Definition, Basic Constraints, and Schema Changes
3. Basic Queries
4. More complex Queries
5. Aggregate Functions and Grouping
6. Advanced Querying Issues
7. Summary of SQL Queries
8. Data Change statements
9. Views
10. Complex Constraints
11. Database Programming

7. Data change statements

- SQL has three commands to change data in a database: INSERT, DELETE, and UPDATE.

7.1 The INSERT command (1)

- In its simplest form, it is used to add *one or more tuples* to a relation.

- Attribute values should be listed *in the same order* as the attributes were specified in the CREATE TABLE command

- Example**:**
  U1:

  INSERT INTO EMPLOYEE
  VALUES ('Richard','K','Marini', '653298653', '30-DEC-52', '98 Oak Forest, Katy, TX', 'M', 37000,'987654321', 4);

- It is also possible to insert into a relation *multiple tuples* separated by commas in a single INSERT command.

7.1 The INSERT command (2)

- An alternate form of INSERT specifies explicitly the attribute names that correspond to the values in the new tuple.
- The attributes are listed in the same order as the values are listed in the command.
- Example:
  U1A:

  INSERT INTO EMPLOYEE (FNAME, LNAME, SSN)
  VALUES ('Richard', 'Marini', '653298653');

- Non-listed attributes are set to their *DEFAULT value or to NULL*.
- *Thus, the listed attributes* must include *all attributes with NOT NULL specification and no default value*.

7.1 The INSERT command (3)

- The constraints specified in the DDL commands are *automatically enforced by the DBMS when updates are applied to the database*.
- It is the responsibility of the user to check that any constraints whose check is not enforced by the system are not violated.
- Example**:**
  The following insertion is rejected if *referential integrity constraint checking* is provided by the DBMS.

  INSERT INTO EMPLOYEE (FNAME, LNAME, SSN, DNO)
  VALUES (‘Robert'**,** ‘Hatcher'**,** ‘980760540‘, 2**)**;

  The following insertion is rejected if *NOT NULL checking* is provided by the DBMS.

  INSERT INTO EMPLOYEE (FNAME, LNAME, DNO)
  VALUES (‘Robert'**,** ‘Hatcher'**,** 5**)**;

7.1 The INSERT command (4)

- Another variation of INSERT allows insertion of *multiple tuples* in conjunction with *creating the relation and loading it with the result of a query*.

- Example**:** Suppose we want to create a temporary table that has the name, number of employees, and total salaries for each department.

```
CREATE TABLE  DEPTS_INFO
              (DEPT_NAME    VARCHAR(10),
              NO_OF_EMPS    INTEGER,
              TOTAL_SAL     INTEGER);

INSERT INTO DEPTS_INFO (DEPT_NAME, NO_OF_EMPS,
                        TOTAL_SAL)
SELECT      DNAME, COUNT(*), SUM(SALARY)
FROM        DEPARTMENT, EMPLOYEE
WHERE       DNUMBER = DNO
GROUP BY    DNAME ;
```

7.1 The INSERT command (5)

- Note: The DEPTS_INFO table may not be up-to-date if we change the tuples in either the DEPARTMENT or the EMPLOYEE relations *after* issuing the insert command. We have to create a view (see later) to keep such a table up to date.

7.2 The DELETE command (1)

- The DELETE command removes tuples from a relation.
- It includes a WHERE-clause to select the tuples to be deleted.
- Tuples are deleted from only *one table* at a time (unless CASCADE is specified on a referential integrity constraint).
- The number of tuples deleted depends on the number of tuples in the relation that satisfy the WHERE-clause.
- Example:

```
DELETE FROM      EMPLOYEE
WHERE   LNAME='Brown' ;
```

7.2 The DELETE command (2)

- Example:
  The following DELETE command will delete at most one tuple. Why?

```
DELETE FROM      EMPLOYEE
WHERE   SSN='123456789' ;
```

- Example:
  The following DELETE command deletes multiple tuples from the EMPLOYEE relation (and possibly from other relations). Why?

```
DELETE FROM      EMPLOYEE
WHERE   DNO  IN
                (SELECT    DNUMBER
                FROM       DEPARTMENT
                WHERE      DNAME='Research');
```

7.2 The DELETE command (3)

- A missing WHERE-clause specifies that all the tuples of the relation are to be deleted.
- Example**:**

DELETE FROM EMPLOYEE ;

- Note that EMPLOYEE table remains in the database as an empty table.

7.3 The UPDATE command (1)

- The UPDATE command is used to modify attribute values of one or more selected tuples.
- A WHERE-clause selects the tuples to be modified.
- An additional SET-clause specifies the attributes to be modified and their new values.
- Each command modifies tuples *in the same relation.*
- Referential integrity should be enforced
- Example**:** The following UPDATE command changes the location and controlling department number of project number 10 to ‘Bellaire’ and 5, respectively.

```
UPDATE    PROJECT
SET       PLOCATION = 'Bellaire', DNUM = 5
WHERE     PNUMBER=10;
```

7.3 The UPDATE command (2)

- Example: Give all employees in the 'Research' department a 10% raise in salary.

```
UPDATE  EMPLOYEE
SET     SALARY = SALARY *1.1
WHERE   DNO  IN
            (SELECT   DNUMBER
            FROM      DEPARTMENT
            WHERE     DNAME='Research');
```

- In this request, the modified SALARY value depends on the original SALARY value in each tuple.

- The reference to the SALARY attribute on the right of = refers to the old SALARY value before modification.

- The reference to the SALARY attribute on the left of = refers to the new SALARY value after modification.

8. Views in SQL (1)

- The relations in databases we have discussed up to this point are more technically referred to as **base relations** (or **base tables**).

- These are "normal" relations.

- The content of a base table is *physically stored on disk* and *is independent of the contents of other tables in the database*.

- The external schema of a database is defined using **views**.

- A **view** is a single *table*. In many respects, a view is like an ordinary table: you can query it, modify it and control access to it.

- However, in several important ways *a view is not a base table*. For one thing: *the content of a view is derived from tables (and possibly other views) stored in the database*.

8. Views in SQL (2)

- A view does not necessarily exist in physical form; it is *usually* a **virtual table** in contrast to base tables whose tuples are actually stored in the database.

- When a view is a virtual table it can be seen as a *query with a name*.

- This limits the possible update operations that can be applied to views.

- There are no limitations on querying a view.

8. Views in SQL (3)

- The role of views in query languages is similar to that of *subroutines* in conventional programming languages.

- A view can be thought of as a way for specifying a table that we need to reference frequently or one that has an independent interest.

- In any case, it makes sense to abstract the query defining the view and "pretend" that the database contains a table which coincides with the result of the query.

8.1 View specification in SQL (1)

- The **CREATE VIEW** command is used to specify a view by specifying a view name, a list of attribute names, and a defining query.
- If none of the view attributes result from applying functions or arithmetic operations, we do not have to specify attribute names for the view. The view attribute names can be *inherited from* the attribute names of the tables in the defining query.
- Example**:**

```
CREATE VIEW     WORKS_ON1   AS
    SELECT   FNAME, LNAME, PNAME, HOURS
    FROM     EMPLOYEE, PROJECT, WORKS_ON
    WHERE    SSN=ESSN AND PNO=PNUMBER;
```

WORKS_ON1

FNAME	LNAME	PNAME	HOURS

8.1 View specification in SQL (2)

- Example:

  In the following view, the view attribute names are listed using a one-to-one correspondence with the entries in the SELECT-clause of the defining query.

```
CREATE VIEW   DEPT_INFO (DEPT_NAME, NO_OF_EMPS,
                         TOTAL_SAL) AS
SELECT        DNAME, COUNT(*), SUM(SALARY)
FROM          DEPARTMENT, EMPLOYEE
WHERE         DNUMBER=DNO
GROUP BY      DNAME;
```

DEPT_INFO

DEPT_NAME	NO_OF_EMPS	TOTAL_SAL

8.1 View specification in SQL (3)

- The DBMS is responsible for keeping the view always up-to-date if the base tables on which the view is defined are modified.

- A view is removed using the **DROP VIEW** command.

- Example**:**

DROP VIEW WORKS_ON1 **;**

DROP VIEW DEPT_INFO **;**

8.2 Queries on views

- We can specify SQL queries on a view in the same way we specify queries involving base tables.

- A view can be defined to simplify frequently occurring queries but also to make complex SQL queries easier to understand and debug.

- Example**:**

  Retrieve the last name and first name of all employees who work on the project named 'ProductX'.

```
SELECT    FNAME, LNAME
FROM      WORKS_ON1
WHERE     PNAME = 'ProductX' ;
```

  Without the view WORKS_ON1, this query specification would require writing two join conditions

8.3 View implementation (1)

- Two main approaches have been suggested for implementing a view: the **query modification approach** and the **view materialization approach**.

8.3 View implementation (2)

- The **query modification** approach equivalently rewrites the query that involves a view into a query on the underlying base tables.

  This approach is also called **virtual view approach** because the (answer of the) view does not have a physical existence.

8.3 View implementation (3)

- Example

Consider the SQL query Q

```
SELECT  FNAME, LNAME
FROM    WORKS_ON1
WHERE   PNAME='ProductX' ;
```

on the view

```
CREATE VIEW WORKS_ON1   AS
SELECT  FNAME, LNAME, PNAME, HOURS
FROM    EMPLOYEE, PROJECT, WORKS_ON
WHERE   SSN=ESSN AND PNO=PNUMBER;
```

8.3 View implementation (4)

- Example (continuation)

Query Q can be rewritten by replacing the view name by the view definition in the FROM clause:

```
SELECT    FNAME, LNAME
FROM      (SELECT FNAME, LNAME, PNAME, HOURS
          FROM    EMPLOYEE, PROJECT, WORKS_ON
          WHERE   SSN=ESSN AND PNO=PNUMBER)
WHERE     PNAME='ProductX';
```

And can also be rewritten equivalently on the base relations without a derived relation in the FROM clause:

```
SELECT    FNAME, LNAME
FROM      EMPLOYEE, PROJECT, WORKS_ON
WHERE     SSN=ESSN AND PNO=PNUMBER AND
          PNAME='ProductX';
```

8.3 View implementation (5)

- The other approach, called **view materialization**, involves physically creating a temporary view table when the view is first queried or earlier.

  When a query that references a materialized view is issued, *it is evaluated using directly the materialized view table*.

  This table is kept on the assumption that other queries on the view will follow.

- If this approach is adopted, an efficient strategy for *automatically updating* the view table when the base tables are updated must be developed in order to keep the view up to date.

8.3 View implementation (6)

- Example

Consider the SQL query Q on the view WORKS_ON1:

```
SELECT  FNAME, LNAME
FROM    WORKS_ON1
WHERE   PNAME='ProductX' ;
```

If WORKS_ON1 is a materialized view, query Q can be computed very efficiently, without any join operation, by a simple lookup on the materialized table WORKS_ON1.

8.3 View implementation (7)

- Usually, if a materialized view is not queried for a certain period of time, the view materialization (that is, the physical view table) is *automatically removed by the system*, and *recomputed from scratch* when future queries reference the view.

  More generally, a *materialized view replacement strategy* needs to be adopted which maximizes the overall performance of the system or guarantees that specific performance requirements are satisfied.

8.3 View implementation (8)

Virtual view approach: advantages and disadvantages

- The **advantage** of the query modification approach is that the views are realized at querying time (if at all).

  Therefore, the views, being virtual, *are always up-to-date*.

- The **disadvantage** of the query modification approach is that *it is often query inefficient for views defined via complex queries* especially if multiple queries are applied to the view within a short period of time.

8.3 View implementation (9)

View materialization approach: advantages and disadvantages

- The **advantage** of the view materialization approach is that *it is, in general, query efficient especially if the views are defined via complex queries* and multiple queries are applied to the view within a short period of time.

- One **disadvantage** of the view materialization approach is that materialized views consume disk space (not a big deal—disk space is cheap).

  A **more important disadvantage** is that *the materialized views need to be updated when the base relations on which they are defined are updated*.

  This incurs an extra cost for the system (*view maintenance cost*).

8.4 Materialized Views (1)

- Materialized views are especially important in **data warehousing**.
- A **data warehouse** is a database that consists of *complex materialized views* defined over data stored in separate (usually remote) operational databases.
- As materialized views become more important, commercial DBMSs are beginning to provide support for such views.

8.4 Materialized Views

- **Maintenance of materialized views** involves the following main considerations:
  - *Materialization time (build method)* determines the time when the view is actually materialized with respect to the creation of the view.
  - *Maintenance policy (refresh mode)* determines the timing when the view is recomputed in relation to base relation updates.
  - *Maintenance strategy (refresh method)* determines the way the materialized views are recomputed. *Incremental view maintenance techniques* is one possibility.

8.5 Updating of Views (Updates on Views)

- Updating of views is complicated and *can be ambiguous*.

- In general, an update on a view defined on a *single table without any aggregate functions* can be mapped to an update on the underlying base table *under certain conditions*.

- If a view involves joins, an update operation may be mapped to update operations on the underlying base relations in *multiple ways*.

  Hence, it is often not possible for the DBMS to determine which update is intended by the user.

- In the following we will consider that views are not updatable.

9. Complex constraints (0)

- Suppose that we want to express the constraint: every PROJECT has at least one employee working on it.

- This constraint *cannot* be expressed by defining the attribute PNUMBER of PROJECT as a foreign key that refers to attribute PNO of WORKS_ON (why?).

- This type of constraint is called *inclusion dependency* and generalizes foreign key constraints.

- This type of constraint and other complex constraints can be expressed using CHECK constraints and ASSERTIONS in SQL.

9. Complex constraints (1)

- In SQL, users can specify more general constraints via declarative assertions, using the CREATE ASSERTION statement of the DDL.
- Each assertion is given a constraint name, and is specified via a condition similar to the WHERE clause of an SQL query.
- Example**:** The constraint "the salary of an employee must not be greater than the salary of the manager of the department that the employee works for" can be specified as follows:

```
CREATE ASSERTION SAL_CONST
CHECK    (NOT EXISTS
    (SELECT *
    FROM     EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
    WHERE    E.SALARY > M.SALARY AND
             E.DNO = D.DNUMBER AND
             D.MGRSSN = M.SSN) );
```

9. Complex constraints (2)

- The constraint is **violated** whenever some tuples in the database cause the condition of the assertion statement to evaluate to FALSE. The constraint is **satisfied** by a database instance if no combination of tuples in the database instance violates the constraint.

- The constraint name can be used later to refer to the constraint or drop it.

- The DBMS is responsible for ensuring that the condition is not violated.

- Any WHERE clause can be used, but many constraints can be specified using the EXISTS and NOT EXISTS style of conditions.

9. Complex constraints (3)

- The CHECK clause can also be attached to an attribute declaration in a CREATE TABLE statement (**attribute-based CHECK**).

- The condition may refer to the attribute being constraint by using the *name of the attribute* in its expression.

- An attribute-based CHECK constraint is checked whenever any tuple gets a new value for this attribute.

  If the constraint is violated by the new value, then the modification is rejected.

9. Complex constraints (4)

- Example**:**

The declaration of DNUMBER attribute of relation DEPARTMENT could be:

DNUMBER INT **NOT NULL** **CHECK** (DNUMBER <=20),

The declaration of the SEX attribute of relation EMPLOYEE could be:

SEX CHAR **CHECK** (SEX = ‘F’ **OR** SEX = ‘M’)

9. Complex constraints (5)

- To declare *a constraint on the tuples of a single table*, when we define that table with a CREATE TABLE statement, we may add to the list of attributes and key or foreign key declarations one or more CHECK clauses (**Tuple-Based CHECK constraint**).

- The condition can be anything that could appear in a WHERE clause.

- The condition of a tuple-based CHECK constraint is checked every time a tuple is inserted into the relation or a tuple of the relation is updated.
  If the condition is false for that tuple, the insertion or update is rejected.

9. Complex constraints (6)

- Example:

  To enforce the constraint that a department cannot have more than 50 employees we can add the following CHECK clause to the definition of EMPLOYEE relation.

```
CHECK (NOT EXISTS
            (SELECT     DNO
            FROM        EMPLOYEE
            GROUP BY    DNO
            HAVING      COUNT(*) > 50))
```

9. Complex constraints (7)

- Example**:**

Alternatively, in order to enforce the constraint that a department cannot have more than 50 employees we can add the following CHECK clause to the definition of EMPLOYEE relation.

```
CHECK (50 >= ALL   (SELECT      COUNT(*)
                   FROM        EMPLOYEE
                   GROUP BY    DNO))
```

Observe that here:

- we do not used **NOT EXISTS**.
- the grouping attribute (DNO) does not appear in the **SELECT** clause.
- we use the keyword **ALL** to compare 50 to the number of employees in every group.

9. Complex constraints (8)

- Example**:**
  Suppose that the DEPARTMENT relation has an additional attribute DEPT_CREATE_DATE to record the date when the department was created.

  We can add the following constraint at the end of the CREATE TABLE DEPARTMENT statement to ensure that a manager's start date is later that the department's creation date.

  CONSTRAINT DATES
  CHECK (DEPT_CREATE_DATE < MGRSTARTDATE)

9. Complex constraints (9)

- The CHECK clause can also be used in conjunction with the CREATE domain statement to specify constraints on a particular domain.

- Example**:**

  To restrict the values of department numbers to an integer number between 1 and 20 we write:

  CREATE DOMAIN D_NUM **AS** INTEGER
  CHECK (D_NUM > 0 **AND** D_NUM < 21)**;**

9. Complex constraints (10)

Table constraints vs. Assertions

- Table constraints (attribute based and tuple based constraints) are required to hold only if the associated table is not empty.
- A table constraint is checked in SQL only when the associated table is modified (insertion, deletion or update of a tuple).
- Table constraint checking can be implemented more efficiently by the DBMS.

- Assertions are not associated with any one table.
- A tuple-based constraint can involve more than one table.
  In this case it is better to create an assertion.

10. Triggers

- Triggers is an alternative way to enforce constraints.